

BUILD-SUPABASE.md

ANNEXE F — Catalogue officiel des patterns transactionnels PostgreSQL

Version : V1

Statut : Référentiel officiel des modèles transactionnels de Vitala

1. Objectif

Cette annexe définit les modèles transactionnels (Transaction Patterns) autorisés dans Vitala.

Elle fournit un cadre de référence pour concevoir des traitements fiables, cohérents et évolutifs, en particulier lorsque plusieurs tables, domaines ou services sont impliqués.

2. Principes directeurs

Tous les traitements transactionnels doivent respecter les principes suivants :

- Atomicité
- Cohérence
- Isolation
- Durabilité (ACID)
- Idempotence lorsque nécessaire
- Simplicité
- Traçabilité

Une transaction ne doit jamais laisser la base dans un état incohérent.

3. Pattern 1 — Transaction simple

Cas d'utilisation

Une seule opération métier affectant une ou plusieurs tables d'un même domaine.

Exemples

- création d'un profil ;
- modification d'un Flash ;
- suppression logique d'une notification.

Règles

- transaction courte ;
 - un seul domaine ;
 - validation complète avant commit.
-

4. Pattern 2 — Transaction multi-tables

Cas d'utilisation

Une opération nécessite la mise à jour de plusieurs tables liées.

Exemples

- création d'une mission ;
- affectation des participants ;
- création des journaux d'audit associés.

Règles

- un seul commit ;
 - aucune mise à jour partielle ;
 - rollback automatique en cas d'échec.
-

5. Pattern 3 — Transaction multi-domaines

Cas d'utilisation

Une opération implique plusieurs domaines fonctionnels.

Exemples

- Mission + Notifications ;
- Flash + Media ;
- UDI + Trust Score.

Règles

- limiter le couplage ;
 - privilégier les événements lorsque cela est possible ;
 - documenter les dépendances.
-

6. Pattern 4 — Saga

Cas d'utilisation

Une opération distribuée ne peut pas être réalisée dans une seule transaction.

Exemples

- création d'un compte ;
- envoi d'un e-mail ;
- génération d'un avatar ;
- synchronisation externe.

Règles

- découpage en étapes ;
 - compensation documentée ;
 - suivi de l'état d'avancement.
-

7. Pattern 5 — Compensation

Cas d'utilisation

Une étape réussit, puis une étape suivante échoue.

Exemples

- suppression d'un média annulée ;
- annulation d'une invitation ;
- échec d'une synchronisation.

Règles

- opération de compensation explicite ;
 - journalisation ;
 - cohérence finale garantie.
-

8. Pattern 6 — Traitement asynchrone

Cas d'utilisation

Une opération longue ne doit pas bloquer l'utilisateur.

Exemples

- génération d'un rapport ;

- traitement d'une image ;
- calcul d'un score ;
- indexation.

Règles

- utilisation de files d'attente ou de workers ;
 - retour rapide à l'utilisateur ;
 - suivi de l'exécution.
-

9. Pattern 7 — Event Driven

Cas d'utilisation

Une action déclenche plusieurs traitements indépendants.

Exemples

- création d'un Flash ;
- notification des abonnés ;
- mise à jour des statistiques ;
- génération d'événements d'audit.

Règles

- événements clairement définis ;
 - faible couplage ;
 - consommateurs indépendants.
-

10. Pattern 8 — Synchronisation Offline

Cas d'utilisation

Réconciliation entre les données locales et PostgreSQL.

Exemples

- Flash créé hors connexion ;
- modification d'un profil ;
- synchronisation de médias.

Règles

- identifiants stables ;
- résolution de conflits documentée ;
- idempotence des traitements ;
- journalisation des synchronisations.

11. Gestion des erreurs

Tous les patterns doivent prévoir :

- la détection des erreurs ;
- la journalisation ;
- les mécanismes de reprise ;
- les stratégies de compensation lorsque nécessaires.

Les erreurs ne doivent jamais laisser un état incohérent.

12. Monitoring

Les transactions critiques doivent être supervisées afin de mesurer :

- leur durée ;
- leur taux d'échec ;
- leur fréquence ;
- leur impact sur les performances.

Les indicateurs doivent être intégrés au système de monitoring.

13. Références croisées

Chaque pattern transactionnel doit pouvoir être relié à :

- BUILD-ARCHITECTURE ;
 - BUILD-SUPABASE ;
 - BUILD-SECURITY ;
 - Domain Model ;
 - API Contracts ;
 - Edge Functions.
-

14. Critères de conformité

Un traitement transactionnel est conforme lorsqu'il :

- respecte les propriétés ACID lorsque requises ;
 - applique le pattern adapté ;
 - est documenté ;
 - est testé ;
 - garantit la cohérence des données.
-

15. Conclusion

Le Catalogue officiel des patterns transactionnels constitue la référence de Vitala pour tous les traitements impliquant PostgreSQL.

Il garantit une architecture robuste, cohérente et évolutive, capable de gérer aussi bien les opérations simples que les workflows distribués et les synchronisations complexes.